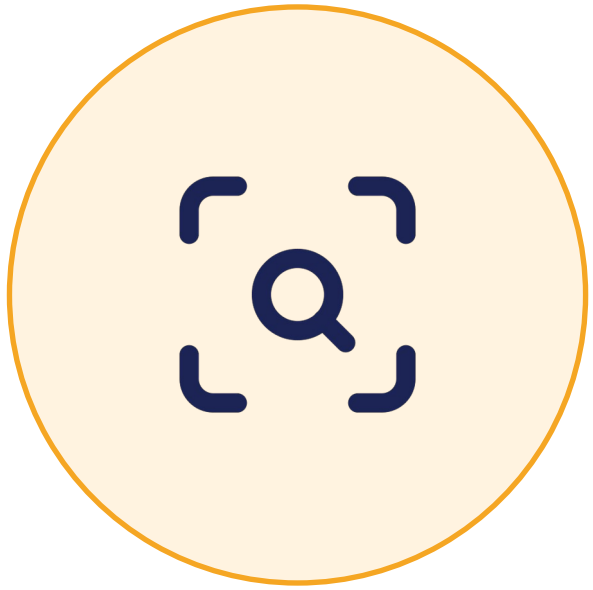
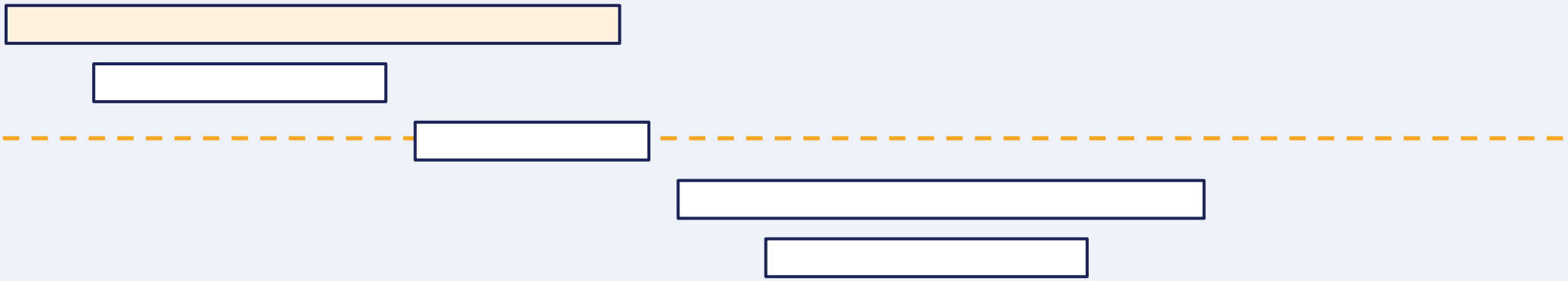
LANGGRAPH · CHAPTER 13 · FLASHCARDS

Observability

Seeing inside every run

11 code cards on LangSmith tracing, from one call to a five-node graph





WHY TRACE · SILENT FAILURE

A wrong answer still returns 200 OK

LLM calls fail as quietly as they succeed, so status codes and uptime cannot catch them

- Monitoring says something is off, observability shows why, evaluation judges whether it was good.
- This chapter builds the "why": a permanent record of every step you can open after the run.

TEXT

```
Traditional software fails loudly
  GET /orders  ->  500 Server Error          (someone gets paged)

An LLM call fails exactly the way it succeeds
  Query:  "What is the capital of France?"
  Status: 200 OK
  Answer: "The capital of France is Berlin."

The chapter's own notebook has one (Experiment 1, saved output)
  Q: "What is a LangSmith Run? Answer in 2 sentences."
  A: "A LangSmith Run is a type of structured exercise or simulation ..."
  -> no exception, no error log, just a wrong answer

Monitoring     what?          latency, error rate, cost, alerts
Observability  why?           prompts, tool calls, steps, timing    <- Ch 13
Evaluation     was it good?   correctness, groundedness, tool choice <- Ch 14
```

Tracing is configuration

Three environment variables switch LangSmith on; no graph code changes at all

- Leave LANGSMITH_PROJECT unset and traces land in a project called "default".
- The older LANGCHAIN_TRACING_V2 / LANGCHAIN_API_KEY names still work: both prefixes are read.

BASH

```
# .env  (add it to .gitignore: a pushed key counts as leaked)
LANGSMITH_TRACING=true                  # the master switch
LANGSMITH_API_KEY=<your LangSmith key>    # authenticates the upload
LANGSMITH_PROJECT=10-observability        # groups traces; created on first use
LANGSMITH_ENDPOINT=https://api.smith.langchain.com    # optional: the default

OPENAI_API_KEY=<your OpenAI key>          # the model provider, kept separate


# Then, in Python, at the very top, before anything traced runs:
#   from dotenv import load_dotenv
#   load_dotenv()          # copies .env into os.environ

# A project is only a container. It never groups a conversation:
# that takes a shared thread_id or session_id (next card).
```

Runs, traces, threads, trajectories

A run is one step, a trace is one invoke(), a thread links traces that share an id

- A lone model call is a run and also the root of its own trace; inside a graph it becomes a nested run.
- A trajectory is not a level above threads: it is the same thread read as one flat list of messages.

TEXT

```
Run         one unit of work: an LLM call, a tool call, a graph node
Trace       the tree of runs from one invoke(), under one root run
Thread      several traces that share a thread_id / session_id
Trajectory  a thread flattened into one ordered list of messages

graph.invoke() #1  ->  Trace 1 --+
graph.invoke() #2  ->  Trace 2 --+--  same thread_id  ->  one thread
graph.invoke() #3  ->  Trace 3 --+

LangGraph copies configurable.thread_id into every run's metadata:
  graph.invoke(inputs, {"configurable": {"thread_id": "demo-1"}})
  root run metadata -> {'thread_id': 'demo-1', 'ls_integration': 'langgraph'}

Long is not the same as threaded: one slow 40-step agent run is still ONE trace.
(metadata checked locally on langgraph 1.2.6 with a callback, nothing uploaded)
```

Every Runnable reports itself

A chat model, a retriever, a tool or a compiled graph is traced with zero tracing code

- run_name changes nothing in the run; it only names the root, which is "LangGraph" when you leave it out.
- The nesting is the call stack: invoke() called the node, and the node called the model.

PYTHON

```python
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.3)
# No LangSmith import: ChatOpenAI is a Runnable, so it traces itself
response = llm.invoke("What is a LangSmith Run? Answer in 2 sentences.")
# -> one LLM run, the root of a one-box trace


# Listings 13.1-13.3: a compiled graph is a Runnable too
def answer_question(state: State):
    response = llm.invoke(f"Give me a brief overview of {state['team']}.")
    return {"answer": response.content}

builder = StateGraph(State)
builder.add_node("answer_question", answer_question)
builder.add_edge(START, "answer_question")
builder.add_edge("answer_question", END)
graph = builder.compile()

result = graph.invoke({"team": "Zamalek SC in Egypt"},
                      config={"run_name": "Zamalek Facts"})   # names the trace
# Zamalek Facts           4.53 s   chain: the root run
# └── answer_question     4.11 s   chain: the node
#     └── ChatOpenAI      4.10 s   llm: the prompt sent, tokens, cost
```

Tracing plain Python

@traceable turns any function into a run; run_type says what kind of box it is

- Common run_type values: "llm", "tool", "chain"; "retriever", "embedding", "prompt" and "parser" also exist.
- Delete the decorator and the search still runs, but its 0.01 s vanishes into the root's total.

PYTHON

```python
from langsmith import traceable

@traceable(run_type="tool", name="doc_keyword_search")
def search_document(query: str, top_k: int = 3) -> list:
    with open("data/llm_production_guide.txt", encoding="utf-8") as f:
        text = f.read()
    paragraphs = [p.strip() for p in text.split("\n\n") if len(p.strip()) > 80]
    keywords = set(re.findall(r"\b\w{4,}\b", query.lower()))
    ranked = sorted(paragraphs, reverse=True,
                    key=lambda p: sum(1 for kw in keywords if kw in p.lower()))
    return ranked[:top_k]


@traceable(run_type="chain", name="doc_qa_pipeline")
def doc_qa(question: str) -> str:
    sections = search_document(question)          # child tool run
    context = "\n\n".join(sections)
    prompt = f"Context:\n{context}\n\nQuestion: {question}\nAnswer concisely:"
    return llm.invoke(prompt).content             # child LLM run, auto-traced

answer = doc_qa("What are the main LLM security threats?")
# doc_qa_pipeline         3.17 s   chain   131 tokens
# ├── doc_keyword_search  0.01 s   tool
# └── ChatOpenAI          3.15 s   llm
```

Attach what you will filter on

get_current_run_tree() reaches the run being built, so you can add metadata before upload

- Arguments land in Input; what you attach lands under Attributes, as a value you can filter on.
- With tracing off there is no run: get_current_run_tree() returns None, so keep the if run: guard.

PYTHON

```python
from langsmith import get_current_run_tree

retriever = vectorstore.as_retriever(search_kwargs={"k": 3})   # FAISS, 27 chunks


@traceable(run_type="chain", name="production_guide_rag")
def rag(question: str, user_id: str = "anonymous") -> str:
    docs = retriever.invoke(question)              # auto-traced retriever run
    context = "\n\n".join(f"[chunk {i+1}] {d.page_content}"
                          for i, d in enumerate(docs))
    prompt = (f"Answer based ONLY on the context below.\n\nContext:\n{context}"
              f"\n\nQuestion: {question}\n\nAnswer concisely:")
    run = get_current_run_tree()
    if run:                                        # None when tracing is off
        run.metadata.update({"user_id": user_id, "chunks_retrieved": len(docs)})
    return llm.invoke(prompt).content              # auto-traced LLM run

rag("How should we evaluate LLM outputs for quality?", user_id="student_02")
# production_guide_rag    1.48 s   Input: question, user_id
# ├── VectorStoreRetriever 0.20 s  embedding the question is a network call
# └── ChatOpenAI           1.27 s
# Attributes -> metadata: chunks_retrieved 3, user_id student_02
```

Making traces queryable

Tags are labels to filter on; metadata is key-value detail, and session_id forms a thread

- LangSmith groups threads on the metadata keys session_id, thread_id or conversation_id.
- run.tags = [...] replaces any tags already on the run; run.add_tags() appends to them instead.

PYTHON

```python
@traceable(run_type="chain", name="support-query")
def support_query(question: str, user_id: str, session_id: str) -> str:
    run = get_current_run_tree()
    if run:
        run.metadata.update({
            "user_id":    user_id,
            "session_id": session_id,       # the thread key
            "feature":    "customer-support",
            "env":        "production",
        })
        run.tags = ["production", "support-bot", "openai"]   # replaces
    return llm.invoke(question).content


for user, session, question in queries:          # sess_001, sess_002, sess_003
    answer = support_query(question, user_id=user, session_id=session)
# Threads view: three single-turn threads, one per session_id
# Filter tag "production": all three at once

# Appending instead of replacing (both are RunTree methods):
#   run.add_tags(["production"]);  run.add_metadata({"env": "production"})
```

Which tools did the agent choose?

A create_agent graph traces itself; compare the tools it called with the ones you expected

- recursion_limit=10 is a hard stop: a looping agent raises GraphRecursionError instead of running on.
- Expected-versus-actual tools is evaluation in miniature; Chapter 14 runs it over many cases.

PYTHON

```python
@tool
def search_local_docs(query: str) -> str:
    """Search the internal LLM production guide. Use only once per question."""
    docs = vectorstore.similarity_search(query, k=3)
    if not docs:
        return "No relevant documents found."
    return "\n\n".join(f"[Chunk {i+1}]\n{d.page_content[:700]}"
                       for i, d in enumerate(docs))[:2500]   # a readable trace

agent = create_agent(model=llm, tools=[search_local_docs, google_search],
                     system_prompt="... maximum two total tool calls ...")


def run_agent(question: str):
    result = agent.invoke({"messages": [HumanMessage(content=question)]},
                          config={"recursion_limit": 10})
    tools_used = [m.name for m in result["messages"] if isinstance(m, ToolMessage)]
    final_answer = next(m.content for m in reversed(result["messages"])
                        if isinstance(m, AIMessage))
    return final_answer, list(dict.fromkeys(tools_used))   # unique, in order

# expected=both   actual=['search_local_docs', 'google_search']
# LangGraph 8.50 s: model 1.27 s -> two tool runs in parallel -> model 5.88 s
```

A graph built with no tracing in mind

Five plain nodes: two call a model, two call a tool, one just joins strings

- A cheap model rewrites the query; the stronger, slower model writes the report, where quality pays.
- steps_taken uses the operator.add reducer, so every node appends its name and the path is recorded.

PYTHON

```python
planner_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)   # cheap
writer_llm = ChatOpenAI(model="gpt-4o", temperature=0)         # strong

class ResearchState(TypedDict):
    query: str
    refined_query: str
    doc_sections: str
    web_results: str
    consolidated_context: str
    final_report: str
    session_id: str
    steps_taken: Annotated[list[str], operator.add]    # appended by each node

def planner(state: ResearchState) -> dict:
    prompt = f"Rewrite this question to be specific and searchable: {state['query']}"
    return {"refined_query": planner_llm.invoke(prompt).content,
            "steps_taken": ["planner"]}

def document_reader(state: ResearchState) -> dict:
    sections = search_local_docs.invoke({"query": state["refined_query"]})
    return {"doc_sections": sections, "steps_taken": ["document_reader"]}

# web_enricher: web_search.invoke(...); synthesizer: an f-string join;
# report_writer: writer_llm.invoke(question + consolidated_context)
```

Where the 20 seconds went

The waterfall splits the total by node, so you optimize the slow step and not the cheap ones

- web_enricher and report_writer take over 90% of the run; document_reader and synthesizer are nearly free.
- load_dotenv() runs before the graph import, because the models and Serper read their keys at import.

PYTHON

```python
builder = StateGraph(ResearchState)
for name, fn in [("planner", planner), ("document_reader", document_reader),
                 ("web_enricher", web_enricher), ("synthesizer", synthesizer),
                 ("report_writer", report_writer)]:
    builder.add_node(name, fn)
builder.add_edge(START, "planner")
# ... planner -> document_reader -> web_enricher -> synthesizer -> report_writer
builder.add_edge("report_writer", END)
graph = builder.compile()

# run_traced.py: the same script you would write without LangSmith
load_dotenv()
from graph import graph
result = graph.invoke(inputs, {"configurable": {"thread_id": "demo-1"}})

# LangGraph (root, no run_name)   ~20 s   4,085 tokens
# ├── planner              1.1 s   ChatOpenAI nested inside
# ├── document_reader      0.3 s   search_local_docs tool run
# ├── web_enricher         6.4 s   web_search tool run: a network call
# ├── synthesizer          0.1 s   nothing nested: plain Python
# └── report_writer       11.8 s   ChatOpenAI (gpt-4o) nested inside
```

Four tracing traps

A late .env, a missing run, a state field mistaken for metadata, a misread run count

- LangSmith caches its settings on the first check, so a late load_dotenv() leaves the whole process untraced.
- "3 runs" in the Runs view counts boxes inside traces; the Traces view counts invoke() calls.

PYTHON

```python
# 1. The LANGSMITH_* settings are read once per process, then cached
doc_qa(question)     # runs before load_dotenv(): tracing is checked -> off
load_dotenv()        # LANGSMITH_TRACING=true is now in os.environ ...
doc_qa(question)     # ... but still untraced. Load .env first, or restart

# 2. Off means no run tree at all: keep the guard
run = get_current_run_tree()
if run:
    run.add_metadata({"user_id": user_id})

# 3. A state field is only Input: this session_id groups nothing
graph.invoke({"query": q, "session_id": "demo-1", "steps_taken": []})
# thread keys go in metadata (or configurable.thread_id for a graph)
graph.invoke(inputs, {"metadata": {"session_id": "demo-1"}})

# 4. Plain Python between Runnables leaves no box until you decorate it

# 1 and 2 checked on langsmith 0.9.8 with nothing uploaded
```